

Here is how to complete the data mutation loop (`POST`, `PUT`, `DELETE`) directly on your **Postgres database**, making full use of the `RETURNING *` clause to match your API's expected lifecycle feedback.

Node.js Lane (Express)

Update your `POST`, `PUT`, and `DELETE` endpoints in `server.js`:

javascript

```
// POST /tasks - Create a task and return the new row with its generated ID
app.post('/tasks', async (req, res) => {
  const { title } = req.body;
  if (!title || title.trim() === "") {
    return res.status(400).json({ error: "Title is required" });
  }

  try {
    const result = await pool.query(
      'INSERT INTO tasks (title, done) VALUES ($1, $2) RETURNING *;',
      [title.trim(), false]
    );
    res.status(201).json(result.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  }
});

// PUT /tasks/:id - Fully update a row with validation and tracking checks
app.put('/tasks/:id', async (req, res) => {
  const taskId = parseInt(req.params.id, 10);
  if (isNaN(taskId)) return res.status(400).json({ error: "Invalid task ID format" });

  const { title, done } = req.body;
  if (title === undefined && done === undefined) return res.status(400).json({ error: "Missing title or done" });
  if (title !== undefined && (!title || title.trim() === "")) return res.status(400).json({ error: "Title is required" });
  if (done !== undefined && typeof done !== "boolean") return res.status(400).json({ error: "Done must be a boolean" });

  try {
    // 1. Fetch current task state to evaluate undefined properties
    const existing = await pool.query('SELECT * FROM tasks WHERE id = $1;', [taskId]);
    if (existing.rows.length === 0) {
      return res.status(404).json({ error: "Task not found" });
    }
    const currentTask = existing.rows[0];

    // 2. Update the task
    const result = await pool.query(
      'UPDATE tasks SET title = $1, done = $2 WHERE id = $3 RETURNING *;',
      [title, done, taskId]
    );
    res.status(200).json(result.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  }
});
```

```

    return res.status(404).json({ error: `Task ${taskId} not found` });
  }

  const finalTitle = title !== undefined ? title.trim() : existing.rows[0].title;
  const finalDone = done !== undefined ? done : existing.rows[0].done;

  // 2. Perform the update structural modification
  const result = await pool.query(
    'UPDATE tasks SET title = $1, done = $2 WHERE id = $3 RETURNING *;',
    [finalTitle, finalDone, taskId]
  );
  res.json(result.rows[0]);
} catch (err) {
  console.error(err);
  res.status(500).json({ error: "Internal server error" });
}
});

// DELETE /tasks/:id - Remove row and return empty 204 layout context
app.delete('/tasks/:id', async (req, res) => {
  const taskId = parseInt(req.params.id, 10);
  if (isNaN(taskId)) return res.status(400).json({ error: "Invalid task ID format" });

  try {
    const result = await pool.query('DELETE FROM tasks WHERE id = $1 RETURNING *;', [taskId]);
    if (result.rows.length === 0) {
      return res.status(404).json({ error: `Task ${taskId} not found` });
    }
    res.status(204).send();
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  }
});

```

Use code with caution.

Python Lane (FastAPI)

Update your `POST`, `PUT`, and `DELETE` routers in your structural setup layout `main.py`:

python

```
from pydantic import BaseModel, Field
from typing import Optional

class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1)

class TaskUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1)
    done: Optional[bool] = None

# POST /tasks - Create a task using structural return references
@app.post("/tasks", status_code=status.HTTP_201_CREATED)
def create_task(task_input: TaskCreate):
    if not task_input.title.strip():
        raise HTTPException(status_code=400, detail={"error": "Title cannot be empty"})

    try:
        with get_db_connection() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO tasks (title, done) VALUES (%s, %s) RETURNING *;",
                    (task_input.title.strip(), False)
                )
                new_task = cur.fetchone()
                conn.commit()
                return new_task
    except Exception as e:
        raise HTTPException(status_code=500, detail={"error": str(e)})

# PUT /tasks/{id} - Safely mutate entry record fields
@app.put("/tasks/{id}")
def update_task(id: int, task_input: TaskUpdate):
    try:
        with get_db_connection() as conn:
            with conn.cursor() as cur:
                # 1. Look up data state elements
                cur.execute("SELECT * FROM tasks WHERE id = %s;", (id,))
                row = cur.fetchone()
                if row is None:
```

```

        raise HTTPException(status_code=404, detail={"error": "Task not found"}

    final_title = task_input.title.strip() if task_input.title is not None else
    final_done = task_input.done if task_input.done is not None else row["done"]

    # 2. Complete column values configuration change
    cur.execute(
        "UPDATE tasks SET title = %s, done = %s WHERE id = %s RETURNING *;",
        (final_title, final_done, id)
    )
    updated_task = cur.fetchone()
    conn.commit()
    return updated_task
except HTTPException:
    raise
except Exception as e:
    raise HTTPException(status_code=500, detail={"error": str(e)})

# DELETE /tasks/{id} - Destroy table row configuration pattern
@app.delete("/tasks/{id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_task(id: int):
    try:
        with get_db_connection() as conn:
            with conn.cursor() as cur:
                cur.execute("DELETE FROM tasks WHERE id = %s RETURNING *;", (id,))
                deleted_row = cur.fetchone()
                if deleted_row is None:
                    raise HTTPException(status_code=404, detail={"error": "Task not found"})
                conn.commit()
                return None # Will emit empty content layer body
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail={"error": str(e)})

```

Use code with caution.

Checkpoint Lifecycle Testing

Execute your step-by-step verification process to ensure everything works flawlessly:

bash

1. Create a task (Should respond with 201 Created and an auto-assigned ID, e.g., 4)

```
curl -i -X POST -H "Content-Type: application/json" -d '{"title":"Deploy to Production"}'
```

2. Mark that newly created task (ID 4) as true using PUT (Should return 200 OK)

```
curl -i -X PUT -H "Content-Type: application/json" -d '{"done":true}' http://localhost:3000
```

3. Pull tasks list tracking array to check changes (Should see updated row)

```
curl -i http://localhost:3000/tasks
```

4. Erase task history tracker using DELETE (Should give back 204 No Content with empty b

```
curl -i -X DELETE http://localhost:3000/tasks/4
```

5. Confirm deletion (Should result in a 404 Error tracking code Lookup)

```
curl -i http://localhost:3000/tasks/4
```